

SMART CAMPUS MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

VISHAL K (VTU26695)

10211CS224

Full Stack Application Development

Slot : E2



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF
SCIENCE AND TECHNOLOGY**

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

CHENNAI 600 062, TAMILNADU, INDIA

MAY, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ” Smart Campus Management System” by VISHAL K (VTU26695) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.Hemamalini.S

Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Head of the Department

Dr.Kujani

Professor & Head

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(VISHAL K)

Date:

ABSTRACT

The Smart Campus Event Management System is a full-stack web application developed to simplify and automate the management of campus events such as workshops, seminars, hackathons, and cultural programs in colleges and universities. Traditional event management often involves manual registration, poor communication, and difficulty in tracking participation. This project provides a centralized digital platform to solve these issues efficiently.

The system has two main user roles: Student and Admin. Students can register and log in, browse upcoming events, view event details such as date, venue, department, type, and seat availability, register for events, view their registered events, receive notifications for upcoming events, and submit feedback. This improves student participation and makes event access easier.

Admins can create, update, and delete events, manage upcoming event announcements, monitor registrations, and view analytics such as total registrations, most popular events, and department-wise participation. This helps in better event planning and management.

The backend is developed using Spring Boot, Spring MVC, and Spring Data JPA (Hibernate ORM), while MySQL is used for database management. The frontend is built using React.js with REST APIs connecting the frontend and backend. Additional features like form validation, exception handling, role-based access, and secure admin login improve system reliability and scalability.

Keywords: Artificial Intelligence (AI), Event Portal, Spring Boot, FastAPI, CRUD Operations, Form Validation, Analytics Dashboard

LIST OF FIGURES

1.1	Framework and Technologies description	8
4.1	Three-Tier Architecture of the Smart Campus Event Management System	24
4.2	Level 0 Data Flow Diagram (Context Diagram)	25
4.3	Level 1 Data Flow Diagram (Sub-processes and Data Stores)	27
5.1	Landing Page Hero Section	31
5.2	Event Discovery and Listing Page	31
5.3	Event Details and Available Spots	32
5.4	Student Dashboard Overview	32
5.5	User Authentication / Login Interface	33
5.6	Event Registration Ticket Modal	33
5.7	Administrator Dashboard Overview	34
5.8	Event Creation Interface	34
5.9	Analytics and Reports Dashboard	35

LIST OF TABLES

2.1	Comparison of Event Management Platforms	13
3.1	Summary of System Technology Stack	22

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
USN	University Seat Number

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	3
1.3.1 In-Scope Features and Functionalities	4
1.3.2 Out of Scope	5
1.4 Frameworks and Technologies	6
1.4.1 Frontend Technologies	6
1.4.2 Backend Technologies	7
1.4.3 Database	8
2 Survey of Similar Applications in Market	9
2.1 Eventbrite	9
2.2 Cvent	10
2.3 Meetup	11
2.4 Traditional/Legacy Campus Systems (Google Forms & Spreadsheets)	11
2.5 Summary and Proposed System Advantage	12

2.6	Comparison with Existing Systems	12
3	FRAMEWORK DESCRIPTION	14
3.1	Frontend Implementation	14
3.1.1	Architectural Structure	14
3.1.2	Routing and Navigation	15
3.1.3	API Integration and Data Fetching	15
3.1.4	Data Visualization	16
3.2	Backend Implementation	16
3.2.1	Architectural Pattern	16
3.2.2	Security and Authentication	17
3.2.3	Database Management	18
3.3	Database Implementation	19
3.3.1	Core Database Entities (Schema)	19
3.3.2	Database Integrity and Relationships	21
3.4	Technology Stack Summary	21
4	METHODOLOGIES	23
4.1	Project Architecture	23
4.2	Data Flow Diagrams (DFD)	23
4.2.1	Level 0 DFD (Context Diagram)	25
4.2.2	Level 1 DFD	26
4.3	System Modules and Functional Architecture	26
4.3.1	Authentication and Authorization Module	27
4.3.2	Administrator Module	28
4.3.3	Student Module	28
4.3.4	Analytics and Reporting Module	29

5	RESULTS AND DISCUSSIONS	30
5.1	System Implementation Overview	30
5.2	System Interface Visualizations	31
5.3	Discussion	35
5.3.1	Significance of the Developed System	35
5.3.2	Security and Technical Robustness	36
5.3.3	Limitations and Future Scope	37
6	CONCLUSION AND FUTURE ENHANCEMENTS	39
6.1	Conclusion	39
6.2	Future Enhancements	40
	References	42

Chapter 1

INTRODUCTION

1.1 Introduction

In modern educational institutions, the successful execution of campus events—ranging from academic workshops and technical symposiums to cultural festivals and departmental seminars—plays a pivotal role in shaping the student experience. However, the management of these events is traditionally a complex and labor-intensive process. Organizers often rely on manual procedures, fragmented communication channels, and paper-based registration systems. This disjointed approach frequently leads to administrative inefficiencies, scheduling conflicts, tracking errors, and ultimately, a lack of student engagement. Students, on the other hand, struggle to discover relevant opportunities due to scattered announcements across notice boards, emails, and informal messaging platforms. Recognizing these critical challenges, the **Smart Campus Event Management System** is proposed as a comprehensive, centralized digital solution designed to streamline the entire lifecycle of campus events.

The primary objective of this system is to bridge the communication gap between event organizers (Administrators) and attendees (Students) by providing a unified, secure, and user-friendly web platform. By digitizing the workflow, the system empowers administrators to seamlessly create, update, and manage events, while ensuring real-time tracking of seat availability and attendee registrations. It entirely eliminates the administrative bottlenecks associated with manual record-keeping. For students, the platform serves as an intuitive discovery portal where they can browse upcoming activities, filter events by date, department, or category, and register with just a few clicks. Furthermore, the inclusion of a dedicated dashboard allows stu-

dents to track their registered events, receive timely notifications, and provide valuable post-event feedback.

To achieve a robust and scalable architecture, the Smart Campus Event Management System is developed as a modern full-stack application. The backend infrastructure is powered by Spring Boot, leveraging Spring Data JPA (Hibernate) for reliable data persistence in a MySQL database. Security is a paramount concern; therefore, the system implements stringent role-based access control using Spring Security and JSON Web Tokens (JWT). This ensures that sensitive administrative functionalities—such as event CRUD operations and user management—are strictly protected from unauthorized access. On the frontend, the application utilizes React.js paired with Tailwind CSS to deliver a highly responsive, aesthetically pleasing, and intuitive user interface.

Beyond core operational features, the system is designed to be data-driven. It incorporates an analytics module that visually presents data on event popularity and departmental participation rates using Recharts. By providing these actionable insights, the Smart Campus Event Management System not only simplifies the logistical aspects of hosting campus activities but also equips academic institutions with the tools necessary to evaluate engagement and continuously improve their event offerings.

1.2 Aim of the Project

The primary aim of the **Smart Campus Event Management System** is to design, develop, and deploy a comprehensive, centralized digital platform that streamlines the organization, discovery, and administration of events within an educational institution.

Specifically, the project aims to achieve the following:

- **Digital Transformation of Event Workflows:** To replace fragmented, manual, and paper-based processes with an automated system that handles the complete lifecycle of campus events, from initial creation by administrators to registration and feedback by students.
- **Enhanced Accessibility and Discovery:** To provide students with an intuitive, unified portal where they can easily browse upcoming events, filter them based on their interests (such as department, date, or event type), and secure their participation through real-time registration with seat availability tracking.
- **Secure and Role-Based Access:** To implement a robust security architecture using JSON Web Tokens (JWT) that distinctly separates user roles. This ensures that students have a personalized experience for tracking their activities, while administrators are granted secure access to manage events, oversee registrations, and moderate feedback.
- **Data-Driven Decision Making:** To equip event organizers and institution administrators with graphical analytics dashboards. By visualizing metrics such as event popularity, attendee turnout, and departmental engagement, the system aims to help institutions make informed decisions to improve future event planning.
- **Improved Campus Engagement:** Ultimately, the project aims to foster a more vibrant and connected campus community by ensuring that opportunities for learning, networking, and extracurricular participation are transparently communicated and easily accessible to all students.

1.3 Scope of the Project

The scope of the **Smart Campus Event Management System** is defined by the features, functionalities, and target users it encompasses, as well as the technolog-

ical boundaries within which it operates. The system is designed primarily for use within an academic institution (e.g., a university or college campus) and caters to two primary user roles: **Students** and **Administrators**.

1.3.1 In-Scope Features and Functionalities

The project encompasses the following core modules and capabilities:

1. User Authentication and Authorization:

- Secure registration and login mechanisms for both Students and Administrators.
- Implementation of JSON Web Tokens (JWT) for session management and securing API endpoints.
- Role-based routing on the frontend to ensure users only access dashboards relevant to their permissions.

2. Administrator Module (Event Organizers):

- **Event Management:** Full CRUD (Create, Read, Update, Delete) capabilities for campus events. Admins can specify details such as event title, description, date, department, event type, and maximum seating capacity.
- **Registration Oversight:** Ability to view all student registrations across different events and monitor seat availability in real-time.
- **Feedback Management:** Access to view and manage post-event feedback submitted by attendees.
- **Analytics Dashboard:** Visual representation (charts and graphs) of system data, including overall event popularity and participation rates broken down by department.

3. Student Module (Attendees):

- **Event Discovery:** A searchable and filterable directory of all upcoming events (filterable by date, department, and type).
- **Registration System:** One-click registration for events, subject to real-time seat availability constraints.
- **Personal Dashboard:** A dedicated space to view "My Events" (currently registered events), receive system notifications, and manage their profile.
- **Feedback Submission:** The ability to submit ratings and written feedback for events they have attended.

4. Technical Scope:

- **Frontend:** A single-page application (SPA) built with React.js, utilizing Tailwind CSS for responsive design, React Router for navigation, and Recharts for data visualization.
- **Backend:** A RESTful API architecture built using the Spring Boot framework (Java 17+).
- **Database:** Relational data modeling and storage using MySQL, integrated via Spring Data JPA (Hibernate).

1.3.2 Out of Scope

To maintain project focus and feasibility, the following features are currently considered out of scope for this iteration:

- Integration with external payment gateways for paid events (all events are currently assumed to be free or handled offline).
- Live streaming or hosting of virtual events directly within the platform (the system manages the logistics, not the hosting of the event itself).

- Integration with the university's central academic grading or attendance systems.
- A dedicated native mobile application (iOS/Android), although the web interface is designed to be mobile-responsive.

1.4 Frameworks and Technologies

The Smart Campus Event Management System is developed as a modern, full-stack web application. It leverages a robust set of industry-standard frameworks and technologies to ensure scalability, security, and a responsive user experience. The technology stack is divided into frontend, backend, and database tiers.

1.4.1 Frontend Technologies

The client-side application is designed to be highly interactive and responsive, providing an intuitive interface for both students and administrators.

- **React.js:** A popular JavaScript library used for building dynamic user interfaces. The project utilizes React to create a Single Page Application (SPA), allowing for fast rendering and seamless transitions between different views without reloading the browser. The project is bootstrapped using **Vite** for optimized and rapid development builds.
- **React Router:** This library is implemented to handle routing within the React application, enabling navigation between different components (e.g., Login, Student Dashboard, Admin Dashboard) while keeping the UI in sync with the URL.
- **Tailwind CSS:** A utility-first CSS framework used for styling the application. Tailwind enables rapid UI development by applying pre-existing utility classes directly to HTML elements, ensuring a consistent, modern, and mobile-responsive design across all devices.

- **Axios:** A promise-based HTTP client used to make asynchronous API requests from the frontend to the Spring Boot backend, handling data fetching, user authentication, and form submissions.
- **Recharts:** A composable charting library built on React components. It is used in the Administrator dashboard to visualize analytics, such as event popularity and department participation, through interactive graphs.
- **Node.js (and npm):** The JavaScript runtime environment and package manager required for running the Vite development server and managing frontend dependencies.

1.4.2 Backend Technologies

The server-side infrastructure is built to be secure, scalable, and capable of handling complex business logic related to event management and user authentication.

- **Java (JDK 17+):** The core programming language used for backend development, chosen for its strong typing, object-oriented principles, and extensive ecosystem.
- **Spring Boot:** An extension of the Spring framework that simplifies the setup and development of robust Java applications. It is used to build the RESTful backend API, significantly reducing boilerplate configuration and providing embedded server support (Tomcat).
- **Spring Security:** A powerful and highly customizable authentication and access-control framework. It is utilized to implement role-based access control, ensuring that API endpoints are secured and accessible only to authorized user roles (Student vs. Admin).
- **JSON Web Tokens (JWT):** Used in conjunction with Spring Security for stateless authentication. Upon successful login, a JWT is generated and sent to the

client, which is then used to securely verify the user's identity on subsequent API requests.

- **Spring Data JPA (Hibernate):** A framework that simplifies database access by providing an abstraction layer over JPA (Java Persistence API). It uses Hibernate under the hood to map Java objects (Entities) to relational database tables, facilitating seamless CRUD operations without writing complex SQL queries.
- **Maven:** A build automation and dependency management tool used to manage the Java project's lifecycle, compile code, and resolve external library dependencies.

1.4.3 Database

- **MySQL:** A widely used open-source relational database management system (RDBMS). It serves as the primary data store for the application, persistently storing critical information including user credentials, event details, registration logs, and user feedback.

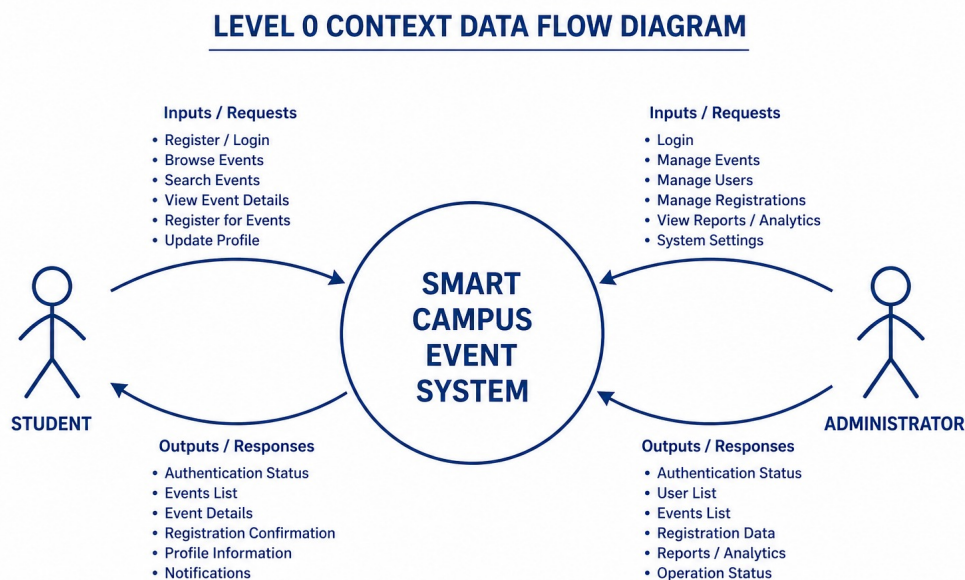


Figure 1.1: Framework and Technologies description

Chapter 2

Survey of Similar Applications in Market

Before developing the Smart Campus Event Management System, it is essential to analyze the existing event management platforms available in the market. While there are numerous robust commercial solutions for general ticketing and event hosting, few are tailored specifically to the unique, closed-ecosystem needs of an academic institution. This chapter reviews several prominent applications, highlighting their features and identifying the gaps that our proposed system aims to fill.

2.1 Eventbrite

Overview: Eventbrite is one of the most popular global platforms for event creation, ticketing, and registration. It caters to a wide variety of events, from small local workshops to massive music festivals.

Key Features:

- Comprehensive ticketing system with paid and free options.
- Built-in marketing and promotional tools.
- QR code generation and mobile check-in capabilities.
- Extensive third-party integrations.

Limitations for Campus Use:

- *Cost and Fees:* Eventbrite charges significant fees for paid events, and its premium features require expensive subscriptions, making it less viable for routine, small-scale college events.
- *Lack of Institutional Integration:* It is a public platform. There is no built-in

way to restrict visibility or registration solely to verified students of a specific university based on their roll numbers or university credentials.

- *Generic Analytics:* While it provides sales data, it lacks the ability to track metrics relevant to a university, such as department-wise participation or academic engagement.

2.2 Cvent

Overview: Cvent is an enterprise-grade event management software primarily targeting large corporate events, conferences, and trade shows.

Key Features:

- Advanced venue sourcing and management.
- Highly customizable event websites and registration workflows.
- Comprehensive attendee tracking and lead generation tools.
- Detailed ROI (Return on Investment) reporting.

Limitations for Campus Use:

- *Overwhelming Complexity:* The platform is overly complex for the straightforward needs of a college tech club or departmental seminar.
- *Enterprise Pricing:* The cost of Cvent is prohibitively high for educational institutions looking for an internal tool.
- *Steep Learning Curve:* The system requires dedicated training to use effectively, which is not suitable for student coordinators who frequently change every semester.

2.3 Meetup

Overview: Meetup is a social networking platform designed to facilitate offline group meetings around shared interests.

Key Features:

- Excellent for community building and discovery based on hobbies or professional interests.
- Group-based architecture where users join a group to see its events.
- Built-in discussion boards and messaging.

Limitations for Campus Use:

- *Informal Structure:* Meetup is inherently informal and lacks the structured registration, seat capacity limits, and formal data collection required for academic events.
- *Data Privacy:* It is a public social network, raising privacy concerns regarding student data and attendance records.
- *No Administrative Oversight:* University administrators cannot centrally monitor or control the events being hosted across different groups on the platform.

2.4 Traditional/Legacy Campus Systems (Google Forms & Spreadsheets)

Overview: In the absence of a dedicated platform, many institutions default to using a combination of Google Forms for registration, WhatsApp/Emails for announcements, and Excel spreadsheets for tracking.

Key Features:

- Free and highly accessible.
- Zero learning curve for basic usage.

Limitations for Campus Use:

- *Highly Fragmented*: Students must check multiple channels to find events.
- *Manual Effort*: Tracking seat limits, sending confirmation emails, and consolidating feedback requires immense manual effort from organizers.
- *Data Inconsistency*: Prone to human error, duplicate registrations, and lost data. There is no unified dashboard to view a student’s extracurricular engagement history.

2.5 Summary and Proposed System Advantage

The survey of existing systems reveals a clear gap in the market. Commercial platforms like Eventbrite and Cvent are too generic, expensive, or complex, lacking the localized control required by universities. Conversely, ad-hoc methods like Google Forms are inefficient and unscalable.

The **Smart Campus Event Management System** is uniquely positioned to bridge this gap. By focusing exclusively on the academic environment, it provides the structured registration of a commercial platform while maintaining the closed, secure, and role-based environment (Student vs. Admin) necessary for a university. It eliminates manual overhead, restricts access to verified campus members, and provides specialized analytics—such as departmental participation—that are critical for institutional assessment, all within a cost-effective, centralized web application.

2.6 Comparison with Existing Systems

The following table summarizes the key differences between existing event management solutions and the proposed Smart Campus Event Management System across various critical parameters relevant to an academic institution.

Table 2.1: Comparison of Event Management Platforms

Feature / Parameter	Eventbrite	Meetup	Google Forms + Sheets	Smart Campus System (Proposed)
Target Audience	General public, commercial events	Hobbyists, social groups	Anyone (Ad-hoc use)	Exclusively Students & Campus Admins
Access Control	Public, Open to all	Public groups	Link sharing (easily leaked)	Strictly restricted via JWT & University IDs
Seat Management	Yes (Automated)	Yes (Automated)	Manual tracking required	Yes (Automated real-time tracking)
Cost	High fees for paid events	Subscription fees for organizers	Free	Free (Internal institutional deployment)
Centralized Discovery	Yes, but cluttered with non-campus events	Yes, but informal	No (Fragmented across WhatsApp/E-mails)	Yes (Unified dashboard for all campus events)
Campus-Specific Analytics	No	No	No	Yes (Departmental participation, event popularity)
Feedback Mechanism	Requires external tools	Comment threads	Separate forms needed	Built-in event rating and feedback module

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

The frontend of the Smart Campus Event Management System is implemented as a Single Page Application (SPA) using React.js. The development environment is bootstrapped using Vite, which provides significantly faster build times and Hot Module Replacement (HMR) compared to traditional bundlers. The user interface is styled utilizing Tailwind CSS, ensuring a responsive, modern, and consistent design language across all devices.

3.1.1 Architectural Structure

To maintain a clean and scalable codebase, the React application follows a modular directory structure. The core logic and UI elements are divided into specific directories within the `src` folder:

- **components/**: Contains reusable, stateless (dumb) components such as buttons, form inputs, modal dialogs, and event cards. These components receive data via *props* and focus solely on the presentation layer.
- **pages/**: Contains stateful (smart) components that represent individual views or routes in the application (e.g., `Login.jsx`, `StudentDashboard.jsx`, `AdminDashboard.jsx`). These components manage page-level state and orchestrate the smaller components.
- **layouts/**: Houses structural components that define the shell of the application, such as the persistent Navigation Bar and Sidebar layouts, ensuring consistent framing for nested page content.

- **context/**: Implements the React Context API for global state management. This is primarily used to manage the authentication state (storing the current user's role, JWT token, and login status) making it accessible throughout the component tree without prop-drilling.
- **services/**: Contains modular files responsible for handling all external HTTP requests. Using Axios, these services encapsulate the logic for communicating with the backend REST APIs, keeping the UI components clean and focused on rendering.
- **utils/**: Stores reusable helper functions, such as date formatting routines and token decoding utilities.

3.1.2 Routing and Navigation

Client-side routing is handled by **React Router DOM**. Instead of requesting new HTML pages from the server, React Router intercepts URL changes and dynamically renders the appropriate React components.

A critical aspect of the frontend implementation is the use of **Protected Routes**. The routing configuration checks the global authentication state (via Context) before rendering certain paths.

- Unauthenticated users attempting to access `/student` or `/admin` routes are automatically redirected to the Login page.
- Role-based protection ensures that a user logged in as a 'Student' cannot navigate to the `/admin` dashboard, preventing unauthorized access at the client level.

3.1.3 API Integration and Data Fetching

Communication with the Spring Boot backend is facilitated through the **Axios** library. To ensure security and handle authentication seamlessly, Axios interceptors

are implemented.

- **Request Interceptor:** Automatically attaches the JWT (stored in local storage) to the Authorization header (e.g., `Bearer <token>`) of every outgoing API request.
- **Response Interceptor:** Catches global errors, such as a `401 Unauthorized` response due to an expired token, and automatically redirects the user to the login screen while clearing stale session data.

3.1.4 Data Visualization

For the Administrator's analytics dashboard, the **Recharts** library is integrated. It consumes aggregated data from the backend APIs (e.g., event participation counts) and renders responsive SVG charts, allowing administrators to visually assess event popularity and departmental engagement metrics.

3.2 Backend Implementation

The backend of the Smart Campus Event Management System is implemented as a robust RESTful API using Java and the **Spring Boot** framework. Spring Boot was chosen for its ability to rapidly develop production-ready, standalone Spring applications with minimal configuration overhead. The backend serves as the core engine, processing business logic, handling secure authentication, and managing data persistence.

3.2.1 Architectural Pattern

The application adheres to the standard **Model-View-Controller (MVC)** architecture adapted for REST APIs, organized into distinct layers to promote separation of concerns, scalability, and maintainability.

- **Controllers (API Layer):** The `Controller` classes define the REST endpoints (e.g., `/api/events`, `/api/auth`). They are responsible for intercepting incoming HTTP requests from the React frontend, mapping them to the appropriate service methods, and returning standardized HTTP responses (JSON payloads and status codes).
- **Services (Business Logic Layer):** The `Service` classes contain the core business logic of the application. They process data, validate inputs (such as checking if an event has available seats before confirming a registration), and act as an intermediary between the Controllers and the Repositories.
- **Repositories (Data Access Layer):** The `Repository` interfaces extend **Spring Data JPA**. They abstract the complex SQL database operations into simple method calls (e.g., `save()`, `findAll()`, `findById()`). Spring Data JPA automatically generates the required Hibernate queries to interact with the underlying MySQL database.
- **Entities (Data Model Layer):** These are Java classes annotated with JPA annotations (e.g., `@Entity`, `@Table`) that map directly to tables in the MySQL database. Key entities in this project include `User`, `Event`, `Registration`, and `Feedback`.
- **DTOs (Data Transfer Objects):** DTOs are utilized to transfer data between the frontend and backend. They prevent the direct exposure of internal database entities to the client, allowing for customized JSON responses and tailored input validation.

3.2.2 Security and Authentication

Security is a critical component of the backend, managed entirely by **Spring Security** integrated with **JSON Web Tokens (JWT)**.

- **Authentication Workflow:** When a user (Student or Admin) attempts to log in via the `/api/auth/login` endpoint, the system verifies their credentials against the database. Upon successful authentication, the backend generates a secure, cryptographically signed JWT containing the user's ID and Role.
- **Stateless Authorization:** The backend is entirely stateless. For every subsequent API request that requires authorization (e.g., registering for an event or creating a new event), the client must include the JWT in the Authorization header. A custom Spring Security `JwtAuthenticationFilter` intercepts the request, validates the token's signature, and sets the authentication context for that specific request.
- **Role-Based Access Control (RBAC):** Endpoints are protected using method-level security annotations (e.g., `@PreAuthorize("hasRole('ADMIN')")`). This ensures that operations like deleting an event or accessing analytics are strictly restricted to users with the 'Admin' role, preventing unauthorized manipulation of data.

3.2.3 Database Management

The system utilizes **MySQL** as its primary relational database. Integration is handled seamlessly via Spring Boot's application properties. **Hibernate**, acting as the JPA provider, manages the automatic creation and updating of the database schema based on the Java Entity definitions (schema-update mode). This approach ensures that the database relationships (such as One-to-Many relationships between Events and Registrations) remain perfectly synchronized with the object-oriented backend code.

3.3 Database Implementation

The Smart Campus Event Management System employs **MySQL**, a robust and widely-used relational database management system (RDBMS), for its data persistence layer. The database schema is designed to efficiently store and retrieve complex, interrelated data concerning users, events, and their interactions.

The interaction between the Java backend and the MySQL database is managed entirely by **Spring Data JPA** and **Hibernate**. Instead of writing raw SQL queries, the database tables are automatically generated and synchronized based on Object-Relational Mapping (ORM). The core domain model consists of five primary entities.

3.3.1 Core Database Entities (Schema)

The database architecture is built upon the following key tables and their relationships:

- **Student Table:** Stores the profiles of the primary end-users. Key fields include a unique identifier (`id`), full name, email address, password (stored as a securely hashed bcrypt string), department, and the University Seat Number (`usn`), which serves as a unique academic identifier.
- **Admin Table:** Maintains the credentials for system administrators and event organizers. Similar to the Student table, it stores an `id`, name, email, and a hashed password. Separating Students and Admins into distinct tables ensures clean role segregation at the database level.
- **Event Table:** Acts as the central entity of the system. It records all details pertaining to a campus event. Key columns include:
 - `id` (Primary Key)
 - `title` and `description`

- `eventDate` and `time`
 - `department` (The department hosting the event)
 - `eventType` (e.g., Workshop, Seminar, Cultural)
 - `totalSeats` (Maximum capacity)
 - `availableSeats` (A dynamic field updated automatically upon new registrations)
- **Registration Table (Junction Table):** This table resolves the Many-to-Many relationship between Students and Events. Every time a student registers for an event, a new record is created here. It contains:
 - `id` (Primary Key)
 - `student_id` (Foreign Key referencing the Student table)
 - `event_id` (Foreign Key referencing the Event table)
 - `registrationDate` (Timestamp of when the booking occurred)

This structure ensures that a student cannot register for the same event multiple times and allows the system to easily query "All events for a specific student" or "All students registered for a specific event".

- **Feedback Table:** Stores post-event reviews submitted by attendees. It includes:
 - `id` (Primary Key)
 - `student_id` and `event_id` (Foreign Keys linking the feedback to a specific user and event)
 - `rating` (Numerical score, e.g., 1 to 5)
 - `comments` (Textual feedback provided by the student)

3.3.2 Database Integrity and Relationships

To ensure data consistency and referential integrity, strict foreign key constraints are established between the tables. For instance, a Registration record cannot exist without a valid `student_id` and `event_id`. Similarly, Cascade operations are configured via JPA annotations to ensure that if an Event is deleted by an Administrator, all associated Registration and Feedback records are subsequently removed to prevent orphaned data in the database.

3.4 Technology Stack Summary

The following table provides a high-level summary of the frameworks, libraries, and technologies utilized across the three tiers of the Smart Campus Event Management System.

Table 3.1: Summary of System Technology Stack

Architecture Tier	Technology / Tool	Primary Purpose in Project
Frontend (Client Tier)	React.js (Vite)	Core JavaScript library for building the single-page user interface (SPA).
	Tailwind CSS	Utility-first CSS framework for rapid, responsive styling and layout.
	React Router	Manages client-side navigation and protected routes based on user roles.
	Axios	HTTP client used to fetch data and send requests to the backend REST API.
	Recharts	React charting library used to generate visual analytics on the Admin Dashboard.
Backend (Application Tier)	Java (JDK 17)	Core object-oriented programming language for server-side logic.
	Spring Boot	Primary framework used to rapidly build the RESTful API and backend services.
	Spring Security	Handles user authentication and Role-Based Access Control (RBAC).
	JWT (JSON Web Tokens)	Provides stateless, secure token-based authentication for API endpoints.
	Spring Data JPA (Hibernate)	Object-Relational Mapping (ORM) tool to interact with the database without raw SQL.
Database (Data Tier)	MySQL	Relational Database Management System (RDBMS) used for persistent data storage.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The Smart Campus Event Management System is designed using a modern **Three-Tier Architecture**. This structural pattern logically separates the application into three distinct computing tiers, ensuring high scalability, maintainability, and security.

- **Presentation Tier (Frontend):** The user-facing layer built with React.js. It runs in the user's web browser and is responsible for rendering the UI, capturing user inputs, and displaying data via dashboards. It communicates with the backend exclusively via RESTful API calls.
- **Application Tier (Backend):** The central logic layer built with Spring Boot. It acts as the intermediary, receiving requests from the frontend, enforcing security rules (JWT validation), executing business logic (e.g., verifying seat availability), and processing data before interacting with the database.
- **Data Tier (Database):** The persistent storage layer utilizing MySQL. It securely stores all application state, including user credentials, event details, and registration logs.

4.2 Data Flow Diagrams (DFD)

Data Flow Diagrams visually represent how information moves through the system, illustrating the inputs, processes, data stores, and outputs. The DFDs for this project are broken down into logical levels of detail.

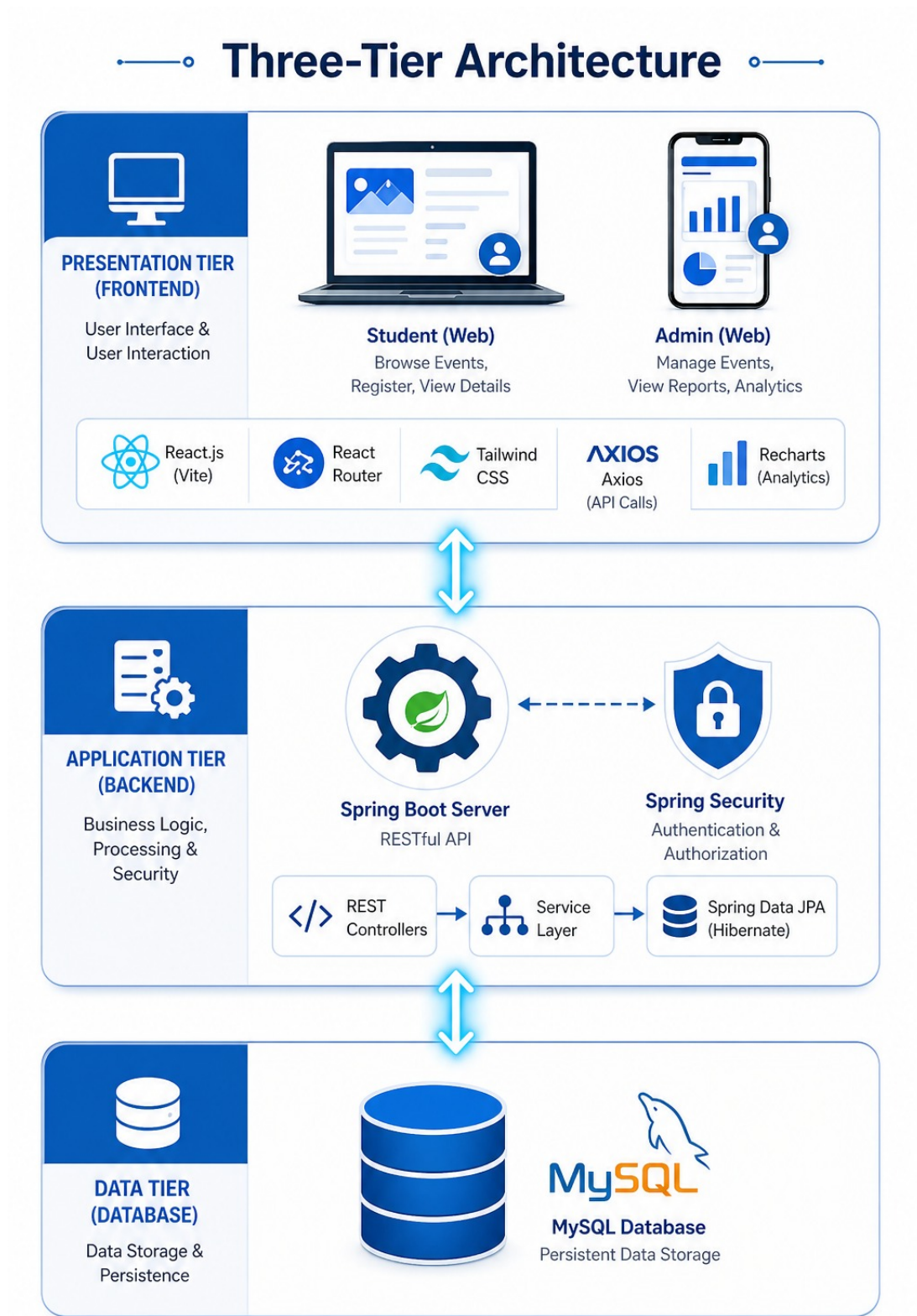


Figure 4.1: Three-Tier Architecture of the Smart Campus Event Management System

4.2.1 Level 0 DFD (Context Diagram)

The Level 0 DFD, also known as the Context Diagram, provides a high-level, abstract view of the entire system. It treats the Smart Campus Event Management System as a single, central process (Process 0) and highlights its interactions with external entities (Users).

External Entities and Data Flow:

- **Student:** Inputs login credentials, registration requests, and feedback into the system. In return, the system outputs event details, registration confirmations, and notifications to the student.
- **Administrator:** Inputs new event details, event updates, and login credentials. The system provides the administrator with registration lists, feedback reports, and analytical data.

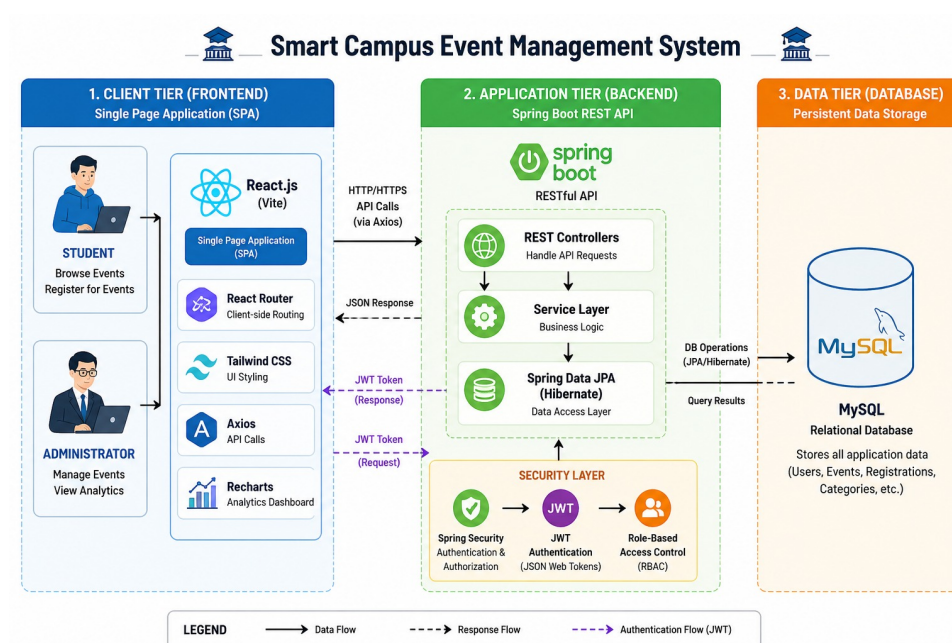


Figure 4.2: Level 0 Data Flow Diagram (Context Diagram)

4.2.2 Level 1 DFD

The Level 1 DFD decomposes the central process from Level 0 into the major sub-processes that make up the system, revealing the internal data stores (Database Tables) and how data is routed between them.

Core Processes in Level 1:

1. **Authentication Process (1.0):** Receives login credentials from Students/Admins. It queries the *User Data Store* to validate credentials. Upon success, it issues a JWT (JSON Web Token) back to the user, granting system access.
2. **Event Management Process (2.0):** Accessible primarily by Administrators. It receives event creation/update data and writes it to the *Event Data Store*. It also outputs the current list of available events to Students.
3. **Registration Process (3.0):** Triggered when a Student attempts to join an event. The process reads the *Event Data Store* to check for seat availability. If seats are available, it records the booking in the *Registration Data Store* and updates the available seat count in the Event Data Store. It then outputs a confirmation to the Student.
4. **Feedback & Analytics Process (4.0):** Receives post-event ratings from Students and writes them to the *Feedback Data Store*. It subsequently aggregates data from the Event, Registration, and Feedback data stores to generate visual analytics and reports for the Administrator.

4.3 System Modules and Functional Architecture

To ensure modularity, scalability, and ease of maintenance, the functional architecture of the Smart Campus Event Management System is logically divided into four primary modules. Each module is responsible for a specific domain of operations

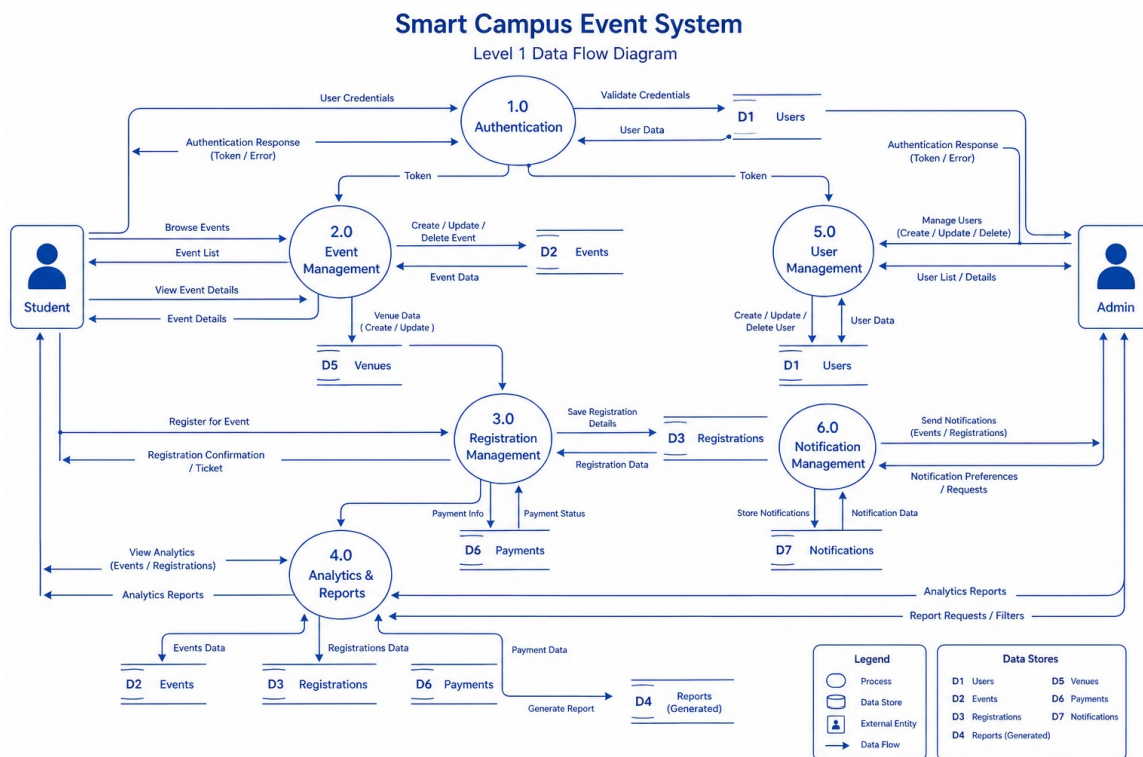


Figure 4.3: Level 1 Data Flow Diagram (Sub-processes and Data Stores)

within the application and interacts seamlessly with the others.

4.3.1 Authentication and Authorization Module

The Authentication and Authorization Module serves as the security gateway for the entire application. Its primary function is to strictly regulate access to the system's resources based on predefined user roles—specifically, 'Student' and 'Admin'. When a new user attempts to access the platform, this module handles the secure registration process, ensuring that sensitive data such as passwords are cryptographically hashed using bcrypt before being stored in the MySQL database.

For returning users, the login process triggers the generation of a JSON Web Token (JWT). This token acts as a digital passport; it is passed between the client and server on every subsequent request to verify the user's identity statelessly. Beyond mere authentication, this module enforces strict Authorization (Role-Based Access Control). It dynamically protects UI routes on the React frontend and secures back-

end API endpoints, ensuring that a user logged in as a student is mathematically blocked from executing administrative commands, thereby maintaining absolute data integrity.

4.3.2 Administrator Module

The Administrator Module is a comprehensive control panel designed exclusively for university event organizers, faculty coordinators, and system administrators. It provides the necessary tools to oversee the entire lifecycle of campus events. Through this module, administrators possess full CRUD (Create, Read, Update, Delete) capabilities. They can seamlessly create new event listings by specifying critical details such as the event title, hosting department, scheduled date, and maximum seating capacity.

Furthermore, the Administrator Module acts as a central hub for logistical oversight. Organizers can view real-time tabular data of all student registrations for any given event, allowing them to anticipate turnout and manage venue logistics effectively. Post-event, this module also serves as a moderation panel where administrators can review ratings and textual feedback submitted by attendees, providing a complete, top-down view of campus engagement.

4.3.3 Student Module

The Student Module forms the core user experience for the general campus populace, focusing heavily on discovery, interaction, and personal organization. Upon logging in, a student is presented with an intuitive Event Discovery Dashboard. This dynamic catalog displays all upcoming campus activities and includes advanced filtering mechanisms, allowing students to narrow down events by specific dates, hosting departments, or categories (such as Technical Workshops or Cultural Fests).

A critical component of this module is the Registration Engine. When a student

attempts to join an event, this engine interacts with the database in real-time to verify seat availability, instantly approving the registration or rejecting it if the event has reached its maximum capacity. Additionally, the module provides a personalized workspace often labeled as "My Events," serving as an academic itinerary where students can track their upcoming commitments. Finally, the module facilitates a feedback loop, allowing students to submit post-event ratings and reviews directly to the organizers.

4.3.4 Analytics and Reporting Module

The Analytics and Reporting Module is a data-driven extension integrated into the Administrator's dashboard, designed to provide actionable, high-level insights. Rather than forcing administrators to manually parse through raw registration tables, this module continuously aggregates live data from the database regarding event frequencies, total registrations, and student engagement metrics.

Utilizing the React Recharts library, this aggregated data is transformed into visually intuitive graphical formats. Administrators can view dynamic bar charts illustrating overall "Event Popularity" or pie charts breaking down "Participation Rates by Department." By transforming raw data into visual analytics, this module significantly aids educational institutions in assessing the success of their extracurricular programs, optimizing future resource allocation, and strategically planning future campus events.

Chapter 5

RESULTS AND DISCUSSIONS

5.1 System Implementation Overview

The Smart Campus Event Management System is designed as a comprehensive, centralized digital platform integrating multiple modules for both students and university administrators.

- Landing Page Interface
- Authentication Module
- Student Dashboard
- Administrator Control Panel
- Event Discovery & Listing Catalog
- Real-Time Registration & Ticketing Engine
- Event Creation & Management Module
- Analytics and Reporting Engine
- Feedback Management System
- Relational Database Layer

The system follows a modular, three-tier architecture ensuring scalability, secure role-based access control, and efficient campus-wide communication by digitizing the entire event lifecycle.

5.2 System Interface Visualizations

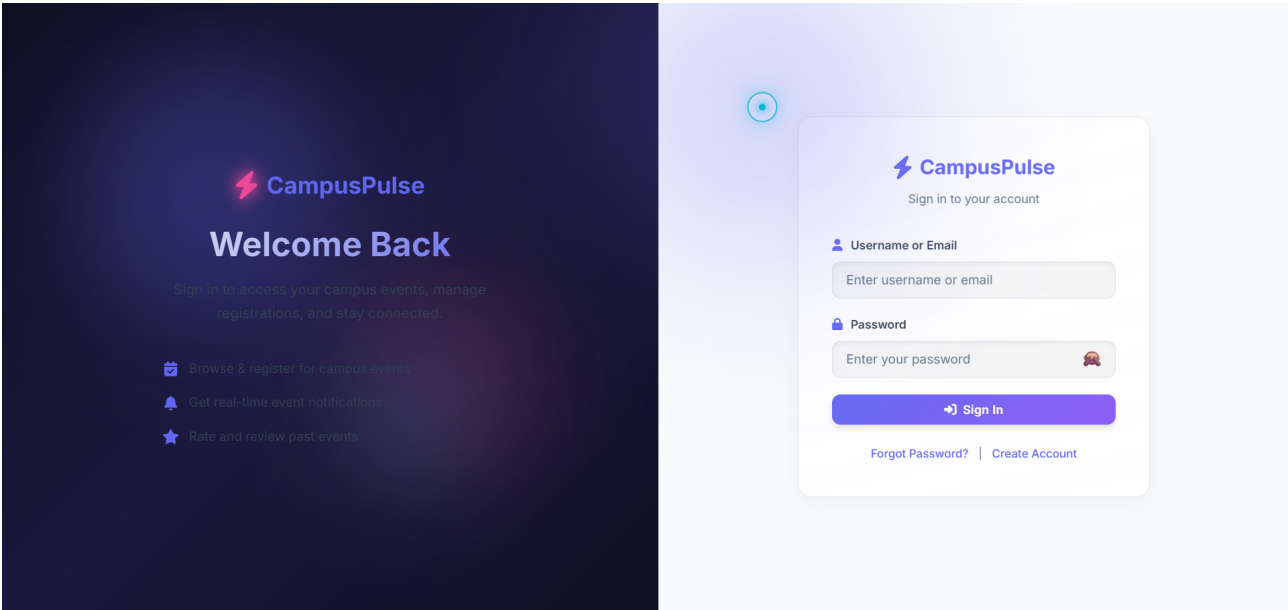


Figure 5.1: Landing Page Hero Section

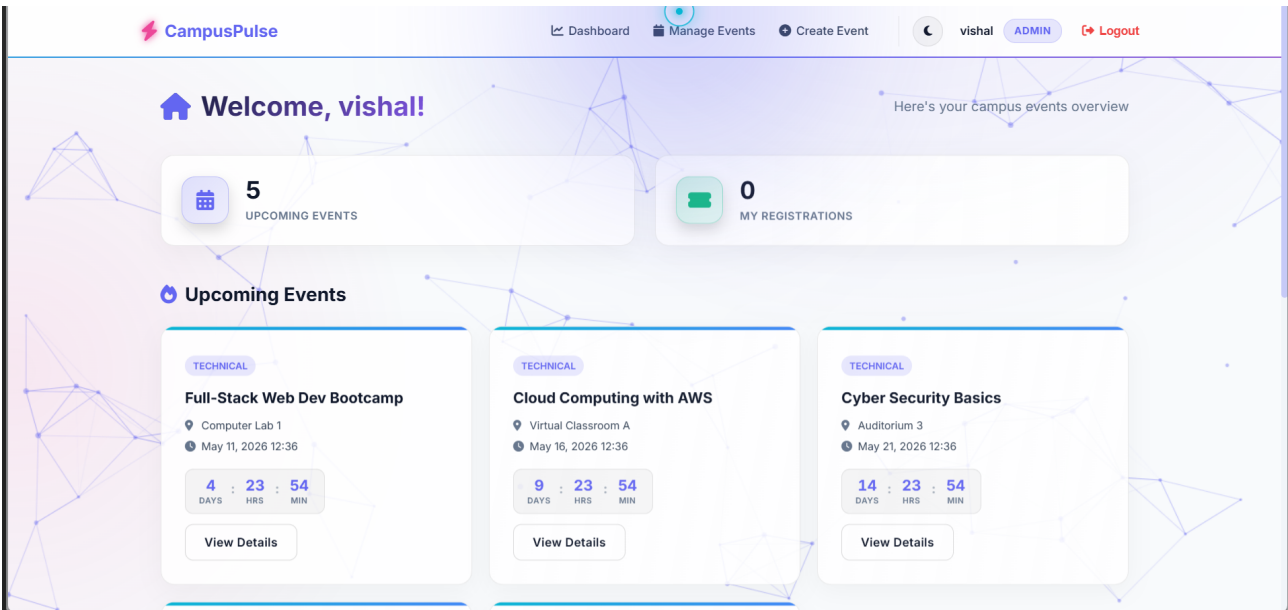


Figure 5.2: Event Discovery and Listing Page

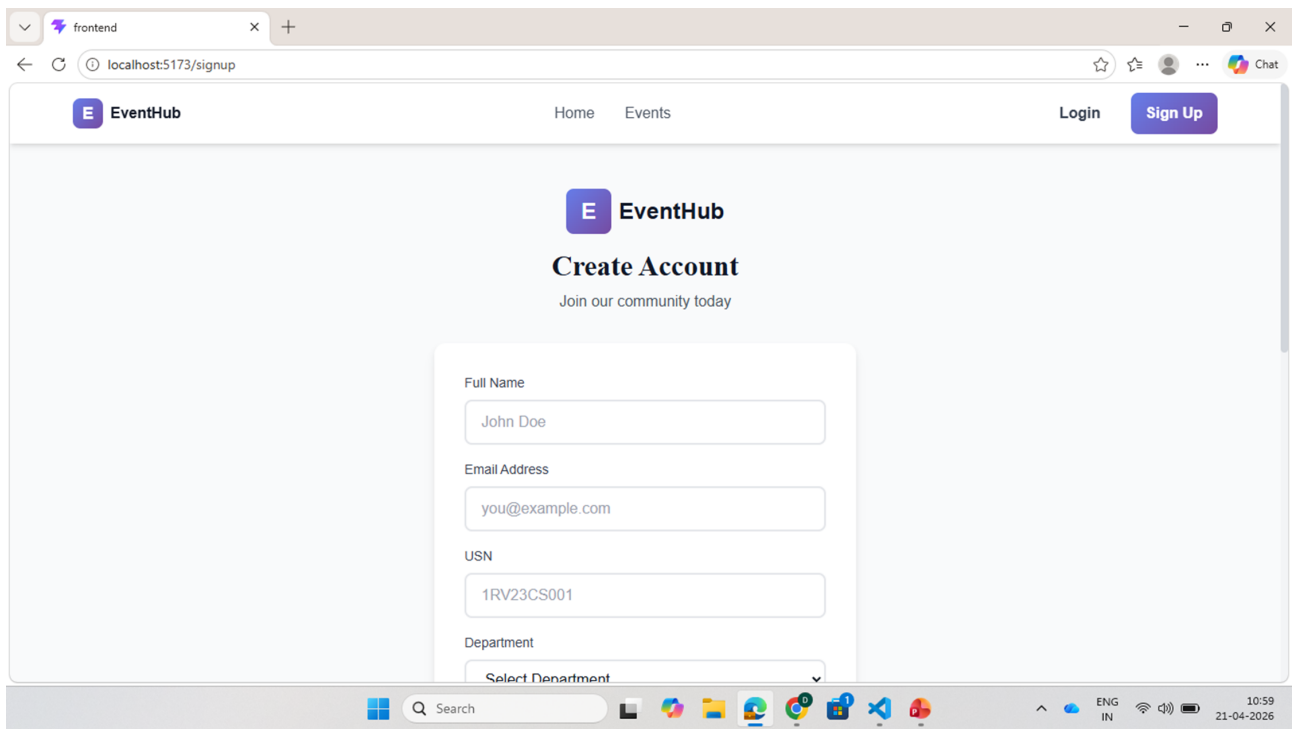


Figure 5.3: Event Details and Available Spots

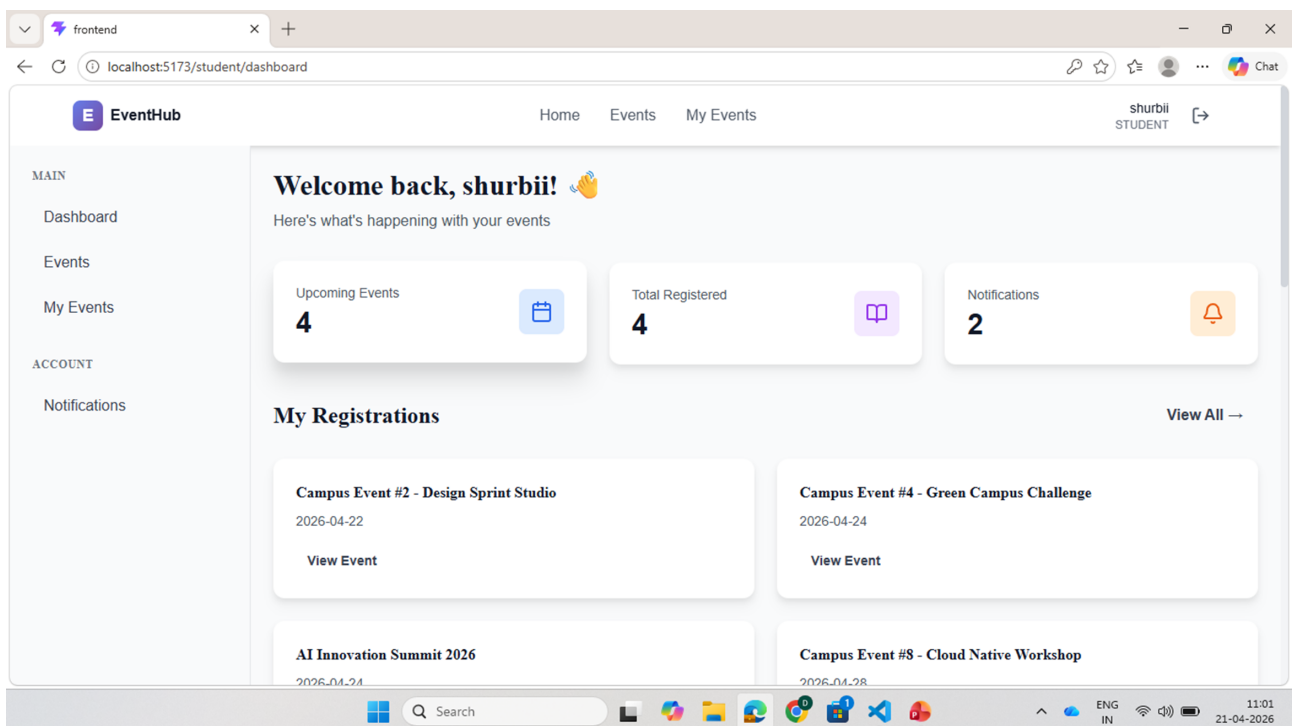


Figure 5.4: Student Dashboard Overview

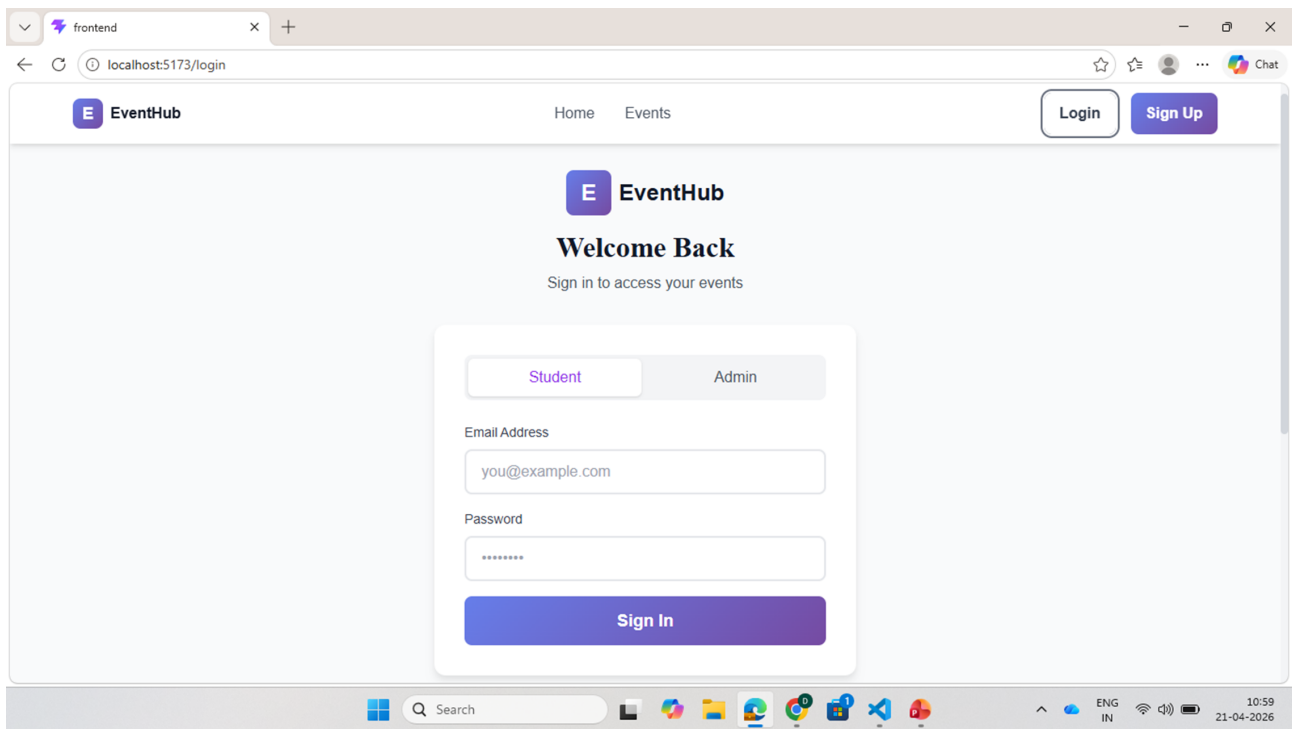


Figure 5.5: User Authentication / Login Interface

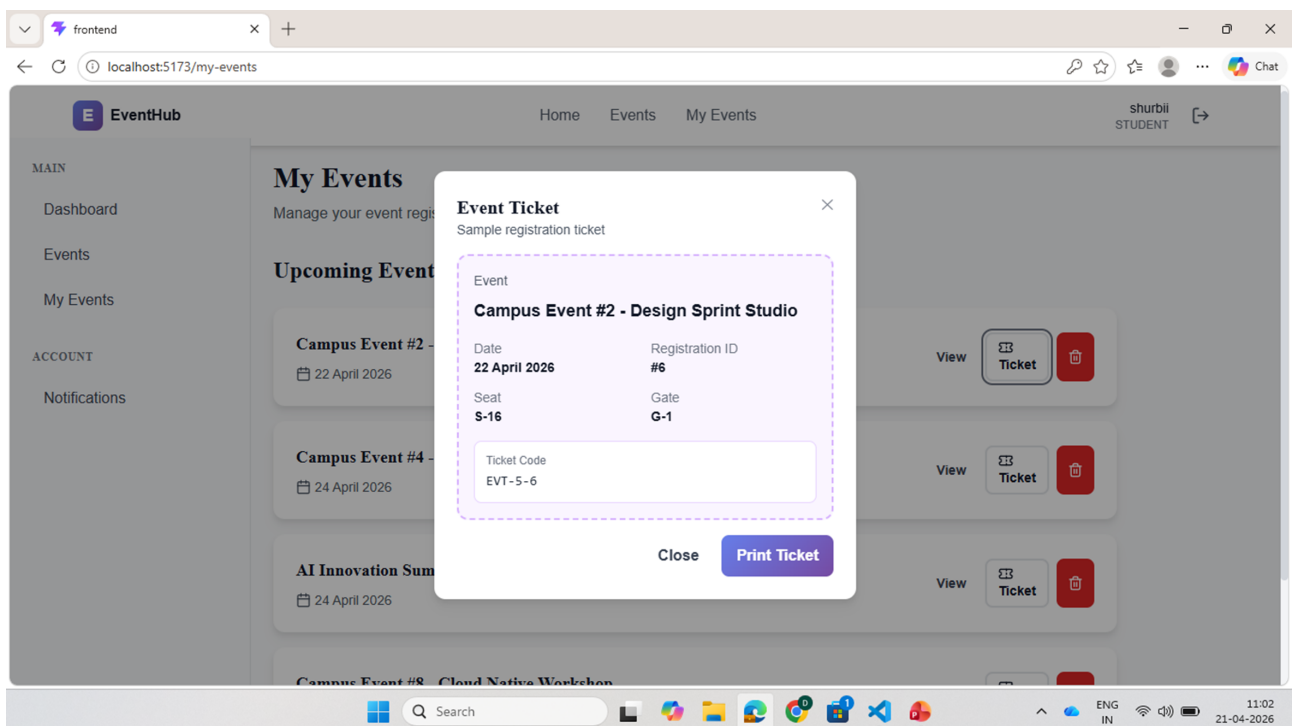


Figure 5.6: Event Registration Ticket Modal

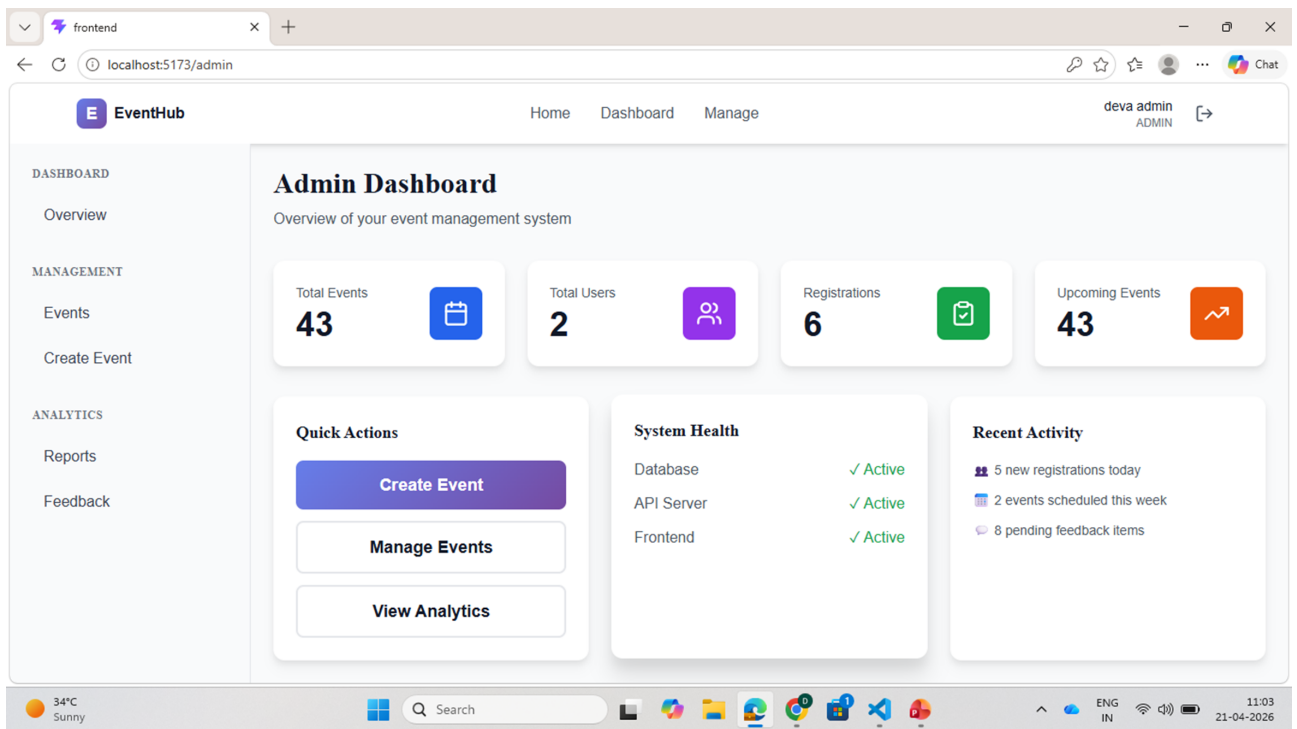


Figure 5.7: Administrator Dashboard Overview

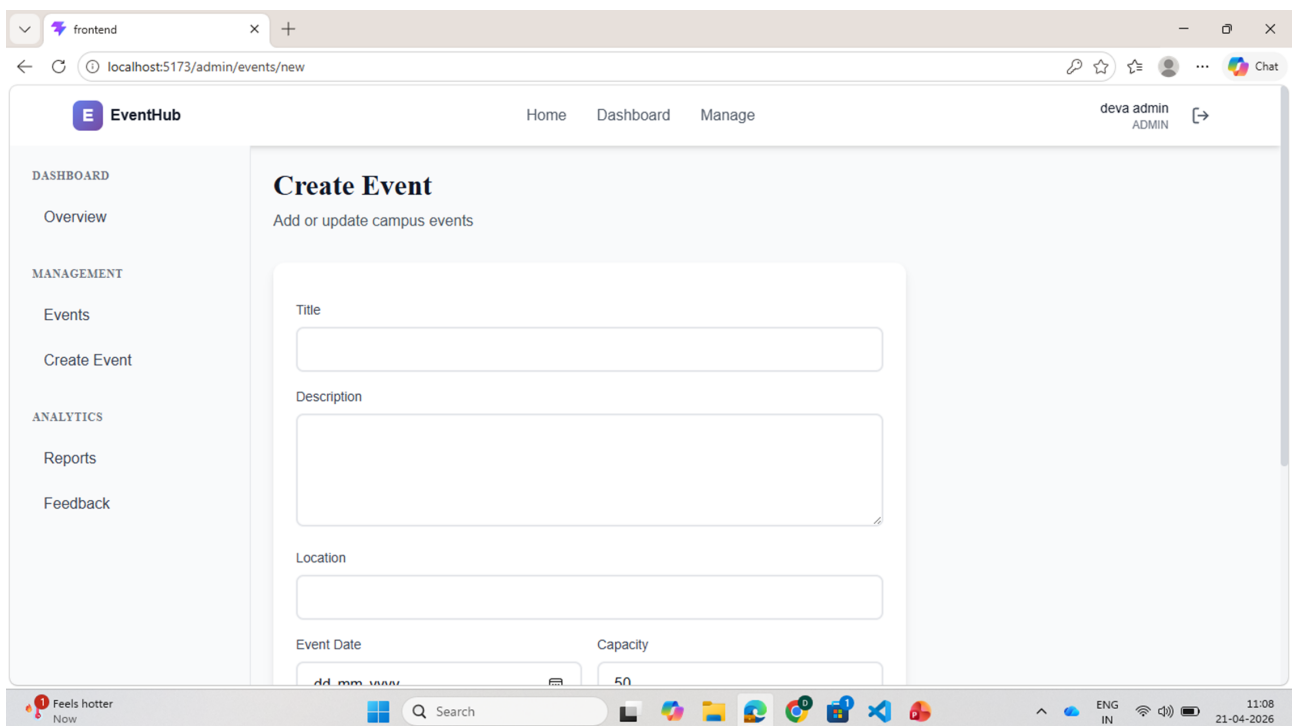


Figure 5.8: Event Creation Interface

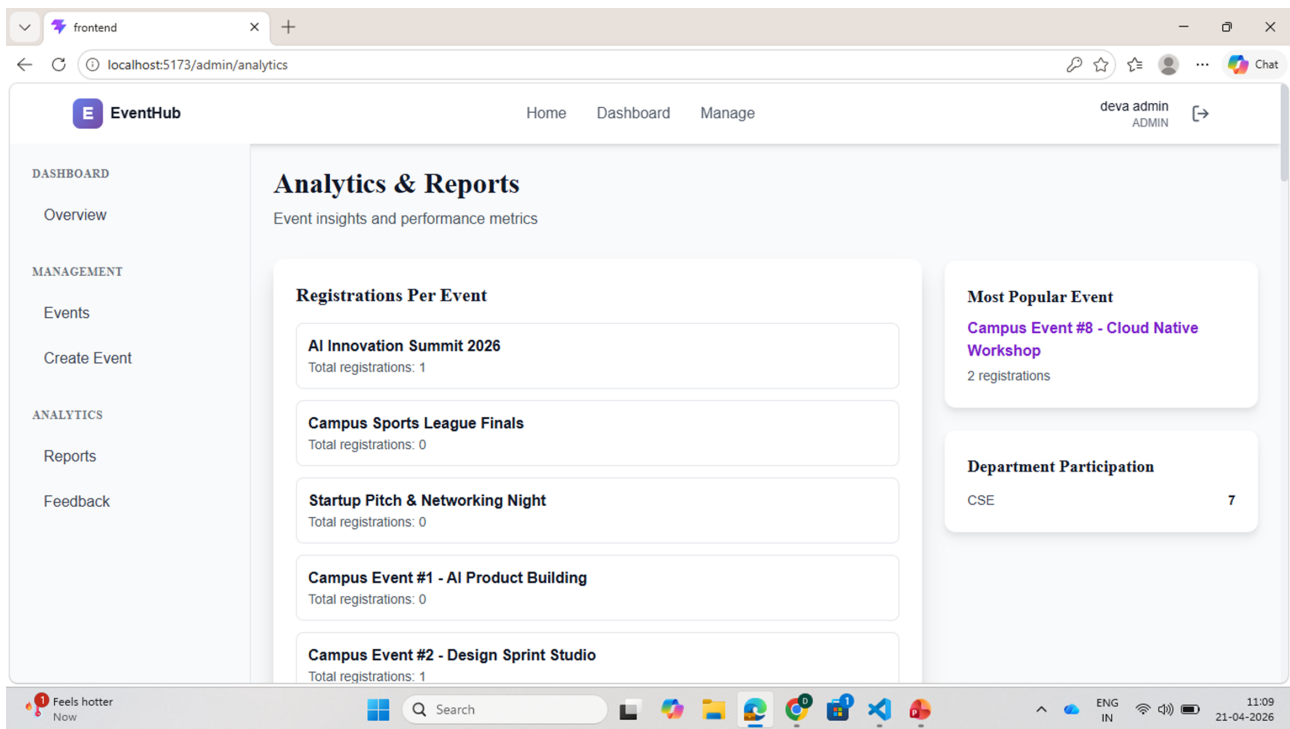


Figure 5.9: Analytics and Reports Dashboard

5.3 Discussion

The development and implementation of the Smart Campus Event Management System represent a significant step toward modernizing the administrative and extracurricular workflows within an academic institution. The primary motivation behind this project was to address the inherent inefficiencies of traditional, fragmented event management methods—such as the reliance on physical notice boards, scattered emails, and ad-hoc spreadsheet tracking—which often led to miscommunication, scheduling conflicts, and poor student engagement.

5.3.1 Significance of the Developed System

The successful deployment of this system demonstrates the substantial advantages of digitizing the campus event lifecycle. By providing a centralized, accessible platform, the system effectively bridges the communication gap between university administrators and the student body.

For students, the platform acts as a singular hub for discovering campus opportunities. The intuitive, filterable event catalog significantly reduces the effort required to find relevant seminars, workshops, or cultural fests. The automated registration engine, coupled with real-time seat availability tracking, provides immediate feedback to the user and eliminates the uncertainty associated with manual registration processes. Furthermore, the personalized "My Events" dashboard fosters better organizational skills, allowing students to seamlessly track their extracurricular commitments.

From an administrative perspective, the platform fundamentally transforms logistical oversight. Event organizers are empowered with comprehensive CRUD capabilities, enabling swift creation and modification of event details. The transition from manual record-keeping to a centralized, relational database (MySQL) minimizes human error, prevents duplicate registrations, and ensures data integrity. Crucially, the integration of the Analytics and Reporting module shifts the institution's approach from reactive to proactive. By visualizing data such as department-wise participation and overall event popularity, decision-makers are equipped with actionable insights to optimize resource allocation and tailor future event planning to student interests.

5.3.2 Security and Technical Robustness

A critical achievement of this project is its robust security architecture. Given that the platform handles sensitive academic identifiers and user credentials, strict separation of privileges was paramount. The implementation of Spring Security utilizing JSON Web Tokens (JWT) ensures that authentication is stateless and highly scalable. The rigid enforcement of Role-Based Access Control (RBAC) guarantees that administrative interfaces and sensitive data endpoints remain entirely inaccessible to student accounts, thereby maintaining the platform's integrity and protecting institutional data.

Furthermore, the adoption of a modular three-tier architecture—leveraging React for a dynamic frontend and Spring Boot for a robust backend—ensures the system remains highly maintainable. This decoupled nature means that future enhancements to the user interface can be made without disrupting the core business logic or database schemas.

5.3.3 Limitations and Future Scope

While the current iteration of the Smart Campus Event Management System successfully fulfills its core objectives, there are certain limitations that provide avenues for future enhancement. Currently, the system assumes all events are free; it lacks integration with external payment gateways to handle paid ticketing or registration fees. Additionally, the platform operates as a standalone web application and is not yet integrated with the university's central academic grading or attendance systems.

Future development could focus on addressing these limitations. Potential enhancements include:

- **Payment Gateway Integration:** Incorporating services like Stripe or Razorpay to facilitate seamless transactions for paid workshops or conferences.
- **Automated Attendance Tracking:** Implementing QR code generation on the digital tickets (as seen in the Registration Modal) and developing a companion mobile scanner app for administrators to streamline on-site event check-ins.
- **Calendar Integration:** Allowing students to sync their registered events directly with Google Calendar or Outlook to prevent personal scheduling conflicts.
- **Advanced Recommendation Engine:** Implementing machine learning algorithms to suggest events to students based on their department, past attendance history, and stated interests.

In conclusion, the Smart Campus Event Management System successfully re-

solves the core challenges of campus event organization. It delivers a scalable, secure, and user-centric solution that not only simplifies administrative overhead but also actively fosters a more engaged and connected campus community.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The development and deployment of the Smart Campus Event Management System mark a crucial advancement in streamlining institutional extracurricular activities. Through the integration of modern web technologies, the project successfully replaces outdated, fragmented processes with a centralized, highly efficient digital ecosystem.

The system effectively meets its primary objective: bridging the communication gap between students and university administrators. By providing students with an intuitive, centralized portal for event discovery and one-click registration, the platform significantly enhances student engagement and accessibility. Simultaneously, it empowers administrators with robust tools to create events, monitor live registration data, and collect actionable feedback without the traditional administrative overhead.

From a technical standpoint, the project demonstrates the efficacy of a decoupled three-tier architecture. The utilization of a React.js frontend ensures a responsive and dynamic user experience, while the Spring Boot backend provides a highly secure, scalable infrastructure. The implementation of stateless JWT authentication alongside strict Role-Based Access Control (RBAC) ensures that institutional data remains protected at all times. Ultimately, the Smart Campus Event Management System delivers a comprehensive, user-centric solution that simplifies logistics, improves data integrity, and fosters a more vibrant and organized campus community.

6.2 Future Enhancements

While the current system fulfills its core requirements, the modular nature of its architecture allows for several promising avenues of future development. To further enhance the platform's utility and user experience, the following enhancements are proposed for subsequent iterations:

- **Integrated Payment Gateway:** Implementing secure, third-party payment gateways (e.g., Stripe, Razorpay) to facilitate seamless, in-app transactions for events that require registration fees or ticket purchases.
- **Automated Attendance and Check-in System:** Upgrading the current digital ticket system to include dynamic QR codes. This would be paired with a mobile-friendly scanner interface for event organizers, enabling rapid, automated on-site attendance tracking.
- **AI-Powered Recommendation Engine:** Integrating machine learning algorithms to analyze a student's past event attendance, department, and stated interests, thereby providing personalized event recommendations directly on their dashboard.
- **Calendar Synchronization:** Developing integration features that allow students to automatically export their registered events to personal calendar applications (e.g., Google Calendar, Microsoft Outlook) to better manage their schedules.
- **Real-Time Notifications Module:** Moving beyond standard dashboard alerts by integrating WebSocket technology or Push Notifications to provide users with instant, real-time updates regarding event cancellations, venue changes, or newly posted events.
- **Dedicated Mobile Application:** While the current web interface is responsive,

developing dedicated native iOS and Android applications would provide a more integrated mobile experience, leveraging native device features like location services and push notifications.

References

- [1] Spring Boot Documentation, VMware, Inc. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>. [Accessed: 30-Apr-2026].
- [2] React Documentation, Meta Platforms, Inc. [Online]. Available: <https://react.dev/>. [Accessed: 30-Apr-2026].
- [3] Spring Security Architecture and JSON Web Tokens (JWT), "Securing a Web Application," Spring Guides. [Online]. Available: <https://spring.io/guides/gs/securing-web/>. [Accessed: 30-Apr-2026].
- [4] Tailwind CSS: Rapidly build modern websites without ever leaving your HTML. [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: 30-Apr-2026].
- [5] MySQL 8.0 Reference Manual, Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Accessed: 30-Apr-2026].
- [6] Hibernate ORM and Spring Data JPA Documentation, Red Hat, Inc. [Online]. Available: <https://hibernate.org/orm/documentation/>. [Accessed: 30-Apr-2026].
- [7] Vite: Next Generation Frontend Tooling. [Online]. Available: <https://vitejs.dev/guide/>. [Accessed: 30-Apr-2026].
- [8] Recharts: A composable charting library built on React components. [Online]. Available: <https://recharts.org/en-US/guide>. [Accessed: 30-Apr-2026].